

A red arrow graphic pointing to the right, located on the left side of the slide, pointing towards the title text.

Les Middlewares Django

L'authentification DJANGO

- Présentation

- L'authentification sous django contient l'authentification proprement dite et l'autorisation
- Elle propose une configuration par défaut
- Le système d'authentification est personnalisable et connectable à des systèmes d'authentification externes
- l'application `django.contrib.auth` et le middleware `django.contrib.auth.middleware.AuthenticationMiddleware` sont en charge de l'authentification, en appui sur les applications et middleware de session.
- La classe `models.User` est au centre des opérations d'authentification

L'Admin DJANGO

- Présentation

- Tout projet Django fournit une interface d'administration automatique
- Elle lit les métadonnées de vos modèles pour fournir une interface rapide et centrée sur les modèles où les utilisateurs de confiance peuvent gérer le contenu du site
- L'application `django.contrib.admin` est installée automatiquement à la création du projet, avec ses dépendances :
 - `django.contrib.auth`
 - `django.contrib.contenttypes`,
 - `django.contrib.sessions`,
 - `django.contrib.messages`
- l'url '`admin/`' est configurée automatiquement
- Il n'est pas conçu pour servir de base à la construction d'une interface publique complète.

L'authentification DJANGO

- L'objet User : attributs
 - username : Obligatoire. Max. 150 caractères. Format : [a-zA-Z0-9_@+.-]
 - first_name : Facultatif. (blank=True). Max. 150.
 - last_name : Facultatif. (blank=True). Max. 150.
 - email : Facultatif (blank=True).
 - Password : Obligatoire. Une empreinte avec métadonnées du mot de passe (Django ne stocke pas le mot de passe en clair). La longueur des mots de passe réels n'est pas limitée, ni les caractères qu'ils contiennent.
 - Groups : clé many-to-many vers le modèle Group.
 - user_permissions : clé many-to-many vers Permission.

L'authentification DJANGO

- L'objet User: attributs
 - is_staff : Valeur booléenne. Indique si cet utilisateur peut accéder au site d'administration.
 - is_active : Valeur booléenne. Indique si cet utilisateur doit être considéré comme actif. définir ce champ à False plutôt que supprimer le compte. Les moteurs d'authentification ne sont pas obligés de vérifier le drapeau is_active,
 - is_superuser : Valeur booléenne. Indique que cet utilisateur possède toutes les permissions sans avoir besoin de les lui attribuer explicitement.
 - last_login : Horodatage de la dernière connexion de l'utilisateur.
 - date_joined : Horodatage indiquant la date de création du compte. Défini par défaut à la date/heure du moment où le compte a été créé.

L'authentification DJANGO

- L'objet User : Méthodes
 - `get_username()` : nom d'utilisateur.
 - `get_full_name()` : `first_name` + ' ' + `last_name`.
 - `get_short_name()` : prénom.
 - `set_password(raw_password)` : Définit le mot de passe de l'utilisateur à la chaîne brute indiquée, en se chargeant du hachage du mot de passe. Lorsque `raw_password` vaut `None`, le mot de passe sera défini comme non utilisable, comme si on avait appelé `set_unusable_password()`.
 - `check_password(raw_password)` : vérifie que le mot de passe est correct.
 - `set_unusable_password()` : Marque l'utilisateur comme n'ayant pas de mot de passe défini.

L'authentification DJANGO

- L'objet User: méthodes
 - `get_user_permissions(obj=None)` : permissions utilisateurs sur un objet ou totale
 - `get_group_permissions(obj=None)` : permissions des groupes utilisateur sur un objet ou totale
 - `get_all_permissions(obj=None)` : permissions utilisateur plus groupes, sur un objet ou totale.
 - `has_perm(perm, obj=None)` : True si l'utilisateur a cette permission. perm est au format "<étiquette application>.<code permission>". False si l'utilisateur est inactif
 - `has_perms(perm_list, obj=None)` : True si l'utilisateur a toutes les permissions en paramètre.
 - `has_module_perms(package_name)` : True si l'utilisateur a au moins une permission dans le module indiqué "<étiquette application>".
 - `email_user(subject, message, from_email=None, **kwargs)` : Envoie un courriel à l'utilisateur. Si `from_email` vaut None, Django utilise `DEFAULT_FROM_EMAIL`. Tout paramètre `**kwargs` sera transmis à l'appel sous-jacent `send_mail()`.

L'authentification DJANGO

- L'objet User: méthodes
 - `get_user_permissions(obj=None)` : permissions utilisateurs sur un objet ou totale
 - `get_group_permissions(obj=None)` : permissions des groupes utilisateur sur un objet ou totale
 - `get_all_permissions(obj=None)` : permissions utilisateur plus groupes, sur un objet ou totale.
 - `has_perm(perm, obj=None)` : True si l'utilisateur a cette permission. perm est au format "<étiquette application>.<code permission>". False si l'utilisateur est inactif
 - `has_perms(perm_list, obj=None)` : True si l'utilisateur a toutes les permissions en paramètre.
 - `has_module_perms(package_name)` : True si l'utilisateur a au moins une permission dans le module indiqué "<étiquette application>".
 - `email_user(subject, message, from_email=None, **kwargs)` : Envoie un courriel à l'utilisateur. Si `from_email` vaut None, Django utilise `DEFAULT_FROM_EMAIL`. Tout paramètre `**kwargs` sera transmis à l'appel sous-jacent `send_mail()`.

L'Admin DJANGO

- Création d'un utilisateur

```
from django.contrib.auth.models import User

# création utilisateur
user = User.objects.create_user('myusername',
                                'myemail@crazymail.com', 'mypassword')

# Modifications
user.first_name = 'John'
user.last_name = 'Citizen'
user.save()
```

L'Admin DJANGO

- Création d'un utilisateur : par l'admin

Change user

Username:

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password: **algorithm:** pbkdf2_sha256 **iterations:** 216000 **salt:** bEldQz***** **hash:** A2TfTT*****

Raw passwords are not stored, so there is no way to see this user's password, but you can change it with a raw password.

Personal info

First name:

Last name:

Email address:

L'Admin DJANGO

- Création d'un utilisateur : permissions

Permissions

☒ Active

Designates whether this user should be treated as active. Unselect this instead of deleting accounts.

☐ Staff status

Designates whether the user can log into this admin site.

☐ Superuser status

Designates that this user has all permissions without explicitly assigning them.

Groups:

Available groups ?

Q

Filter

Chosen groups ?

L'Admin DJANGO

- Les vues d'authentification : configuration

à ajouter dans le fichier d'urls projet

```
path('accounts/', include('django.contrib.auth.urls')),
```

- Le template de login par défaut doit être placé dans un dossier templates/registration/login.html

L'Admin DJANGO

- Les vues d'authentification : configuration

à ajouter dans le fichier d'urls projet

```
path('accounts/', include('django.contrib.auth.urls')),
```

- Le template de login par défaut doit être placé dans un dossier templates/registration/login.html
- La page de login possède les fonctionnalités classiques (forgotten password)
- La page de login redirige par défaut sur accounts/profile
- On peut régler cette redirection avec la variable de configuration **LOGIN_REDIRECT_URL**

L'Admin DJANGO

- Contrôle d'authentification dans les templates
- La variable `{{ user }}` est accessible par défaut dans la configuration de base

```
{% if user.is_authenticated %}
```

```
    <li>User: {{ user.get_username }}</li>
```

```
    <li><a href="{% url 'logout' %}?next={{ request.path }}">Logout</a></li>
```

```
{% else %}
```

```
    <li><a href="{% url 'login' %}?next={{ request.path }}">Login</a></li>
```

```
{% endif %}
```

L'Admin DJANGO

- Contrôle d'authentification dans les vues
 - `request.user` possède l'objet user
 - Les vues fonctions peuvent être décorées avec `@login_required`
 - Les vues classes peuvent rajouter un héritage de `LoginRequiredMixin`

```
@login_required
def my_view(request):
    ...
class MyView(LoginRequiredMixin, View):
    login_url = 'accounts/login/'
    redirect_field_name = 'next'
```

L'Admin DJANGO

- Gestion des permissions
 - Les permissions sont créées dans le code ou dans l'admin
 - Les permissions sont associées aux métas des modèles

```
class BookInstance(models.Model):  
    class Meta:  
        permissions = (("can_read", "access article"),  
                        ("can_publish", "change article status"))
```

L'Admin DJANGO

- Contrôle des permissions dans les templates
- La variable `{{ perms }}` de l'utilisateur est disponible par défaut dans la configuration de base

```
{% if perms.article.can_read %}  
    <!-- SHOW ARTICLE -->  
{% endif %}
```

L'Admin DJANGO

- Contrôle des permissions dans les vues
- Les vues fonctions peuvent être décorées par `@permission_required`
- Les vues classes peuvent rajouter un héritage de `PermissionRequiredMixin`

```
@permission_required('article.can_read')  
def my_view(request):
```

```
class MyView(PermissionRequiredMixin, View):  
    permission_required = 'article.can_publish'  
    # Or multiple permissions  
    permission_required = ('article.can_read', 'article.can_publish')
```

i18n DJANGO

- **Présentation**

- Le but de l'internationalisation et de la régionalisation est de permettre à une seule application Web de présenter son contenu dans des langues et des formats adaptés à ses visiteurs.
- Sur ces sujets, Django effectue deux opérations :
 - Il permet d'indiquer quelles parties des templates doivent être traduites ou mises en forme en fonction des langues et cultures locales.
 - Il utilise ces marqueurs pour régionaliser les applications Web en fonction des préférences des utilisateurs.
- La traduction dépend de la langue cible et la mise en forme dépend généralement du pays cible, informations données par le navigateur dans l'en-tête **Accept-Language**.
- La traduction repose sur l'utilisation de la fonction **gettext()** ou de son raccourci « **_()** »

i18n DJANGO

- gettext

```
def my_view(request):  
    # clé simple  
    output = _("Welcome to my site.")  
  
    # clé cachée  
    sentence = 'Welcome to my site.'  
    output = _(sentence)  
  
    # clé formatée : pratique pour gérer des inversions du au langage  
    def my_view(request, m, d):  
        # Translators: message à desintation des traducteurs  
        output = _('Today is %(month)s %(day)s.') % {'month': m, 'day': d}
```

i18n DJANGO

- Extraction des chaines à traduire
- django-admin **makemessages** :
 - Parcourt la totalité de l'arborescence de code source à partir du répertoire courant et extrait toutes les chaînes marquées pour la traduction.
 - Crée (ou met à jour) un fichier de messages dans le répertoire locale pour les projets et les applications.
 - Après la mise à jour des fichiers de messages, il est encore nécessaire de les compiler par **compilemessages** pour qu'ils soient utilisables par l'outil intégré gettext
- Paramètres de la commande :
 - locale=xx : locale des fichiers créés
 - extensions=html,... : extensions des fichiers à scruter
 - a : scruter la totalité du code source

i18n DJANGO

- Fichiers de traduction

```
#. Translators: message à destination des traducteurs  
# path/to/python/file.py:123  
msgid "Welcome to my site."  
msgstr ""
```

i18n DJANGO

- ngettext : traduction des pluriels

```
count = Article.objects.count()
if count == 1:
    name = Article._meta.verbose_name (« article »)
else:
    name = Article._meta.verbose_name_plural (« articles »)

text = ngettext(
    'There is %(count)d %(name)s available.', # si count == 1
    'There are %(count)d %(name)s available.', # sinon
    count
) % {
    'count': count,
    'name': name
}
```

i18n DJANGO

- traduction dans les templates

- La balise `{% translate %}` traduit son paramètre dans la locale courante

```
<title>{% translate "This is the title." %}</title>  
<title>{% translate myvar %}</title>
```

```
# traduction non affichée, cachée dans une variable  
{% translate "This is the title" as the_title %}
```

- La balise `{% blocktranslate %}` permet de marquer comme chaînes à traduire des phrases complexes contenant à la fois du texte littéral et des variables

```
{% blocktranslate %}This string will have {{ value }} inside.{% endblocktranslate %}
```

- Commentaires

```
{% comment %}Translators: View verb{% endcomment %}  
{% translate "View" %}
```

i18n DJANGO

- pluriels dans les templates
- `{% blocktranslate %}` utilise un mot clé `count` associé à une variable dont on teste la valeur `> 1` pour activer ou non la section `{% plural %}`

```
{% blocktranslate count counter=list|length %}
```

There is only one {{ name }} object.

```
{% plural %}
```

There are {{ counter }} {{ name }} objects.

```
{% endblocktranslate %}
```

i18n DJANGO

- Activation d'une locale dans les vues
- L'appel à **django.utils.translation** fixe la locale des fichiers de traductions appelés

```
from django.utils.translation import activate  
activate('fr')  
...
```

i18n DJANGO

- Activation d'une locale dans les templates
- `{% language %}` active la locale en paramètre

```
{% load i18n %}
```

```
{% get_current_language as LANGUAGE_CODE %}
```

```
<!-- Current language: {{ LANGUAGE_CODE }} -->
```

```
<p>{% translate "Welcome to our page" %}</p>
```

```
{% language 'en' %}
```

```
    {% get_current_language as LANGUAGE_CODE %}
```

```
    <!-- Current language: {{ LANGUAGE_CODE }} -->
```

```
    <p>{% translate "Welcome to our page" %}</p>
```

```
{% endlanguage %}
```